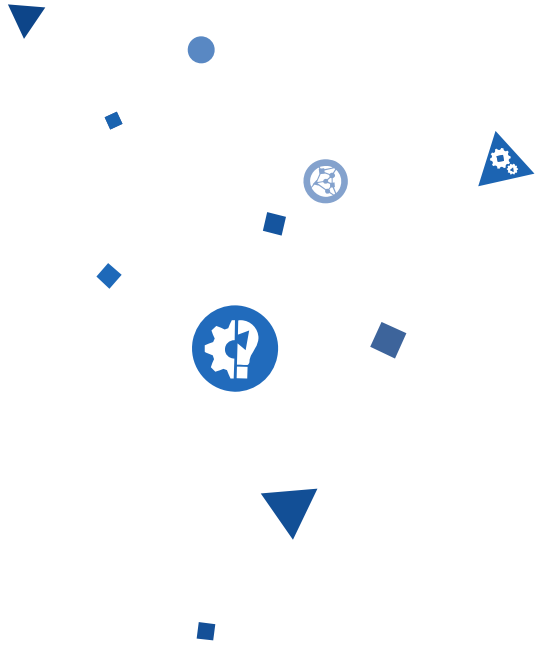


Architecting for performance. A top-down approach.

Ionuţ Baloşin

Software Architect
ibalosin@luxoft.com
@ionutbalosin

About Me

Ionuţ Baloşin

Software Architect @ www.luxoft.com

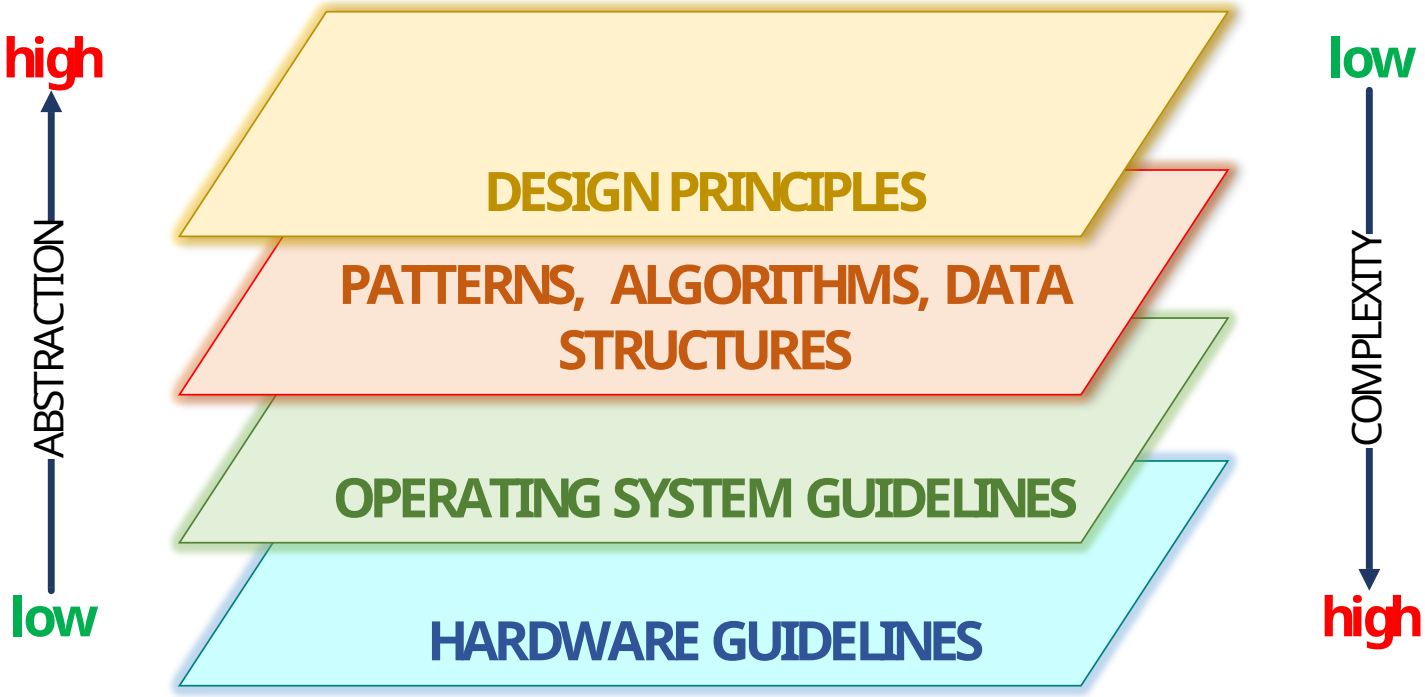
- **10+** years of software development experience

Technical Trainer @ www.luxoft-training.com

- Java Performance and Tuning
- Software Architecture

Speaker, Blogger

Agenda


high
ABSTRACTION
low
DESIGN PRINCIPLES
PATTERNS, ALGORITHMS, DATA STRUCTURES
OPERATING SYSTEM GUIDELINES
HARDWARE GUIDELINES
low
COMPLEXITY
high

Target Audience

Performance Engineers

Software Architects

Developers passionate about Performance

Anybody else interested in the topic ☺

What is Performance?

*"**Performance** it's about **time** and the software system's ability to meet **timing requirements**."*

"Software Architecture in Practice" - Rick Kazman, Paul Clements, Len Bass

https://www.infoq.com/articles/IT-industry-better-namings

Does IT Industry Need Better Namings?

Posted by **Ionut Balosin** , reviewed by **Ben Linders** on Apr 24, 2017.

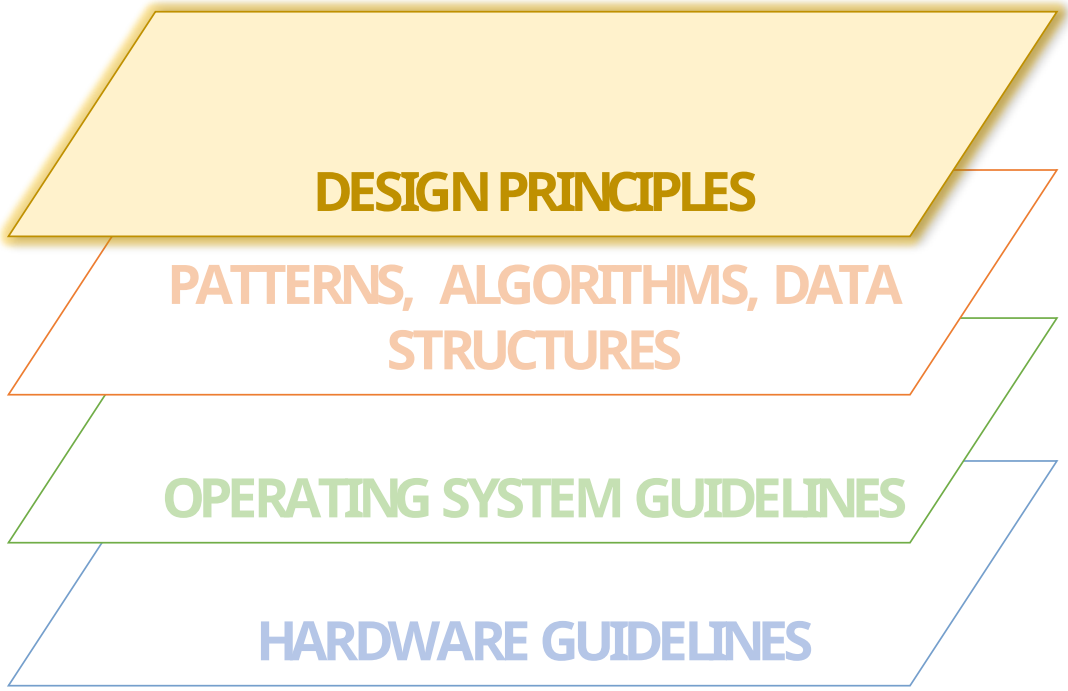
Non-functional requirements or quality attributes

Oftentimes, business requirements are drilled down in functional and non-functional specifications. In this context, the non-functional requirements refer to quality attributes such as performance, availability, scalability, security, usability, etc.

But let's try to better understand what's behind the non-functional terminology guided by two approaches: the first one is to search the word in the dictionary and the second one is to show the technical implications with regards to their achievability.

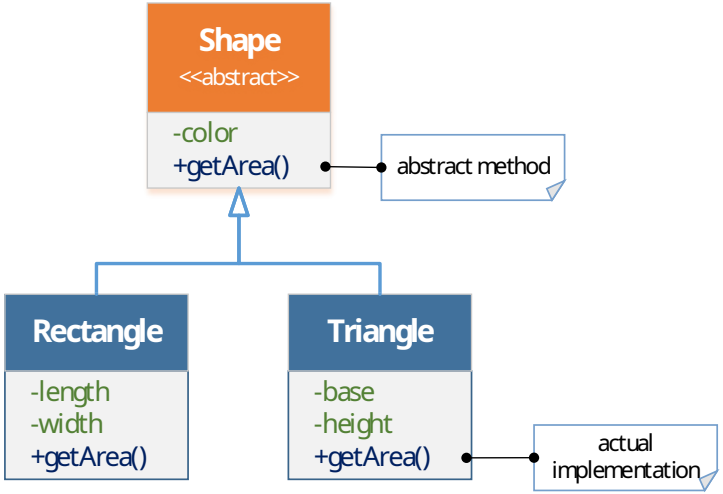
According to the dictionary, the *"non"* word means "*not, absence of, unimportant, worthless*". Based on this definition, the question which arises is: how can we name something non-functional and still having an impact on the architecture, since by semantics it is "worthless"? Why shouldn't we drop or put aside these requirements? Hence, the first inconsistency.

Following with the second approach, all non-functional requirements (e.g. security, usability) are


DESIGN PRINCIPLES
PATTERNS, ALGORITHMS, DATA STRUCTURES
OPERATING SYSTEM GUIDELINES
HARDWARE GUIDELINES

Abstractions

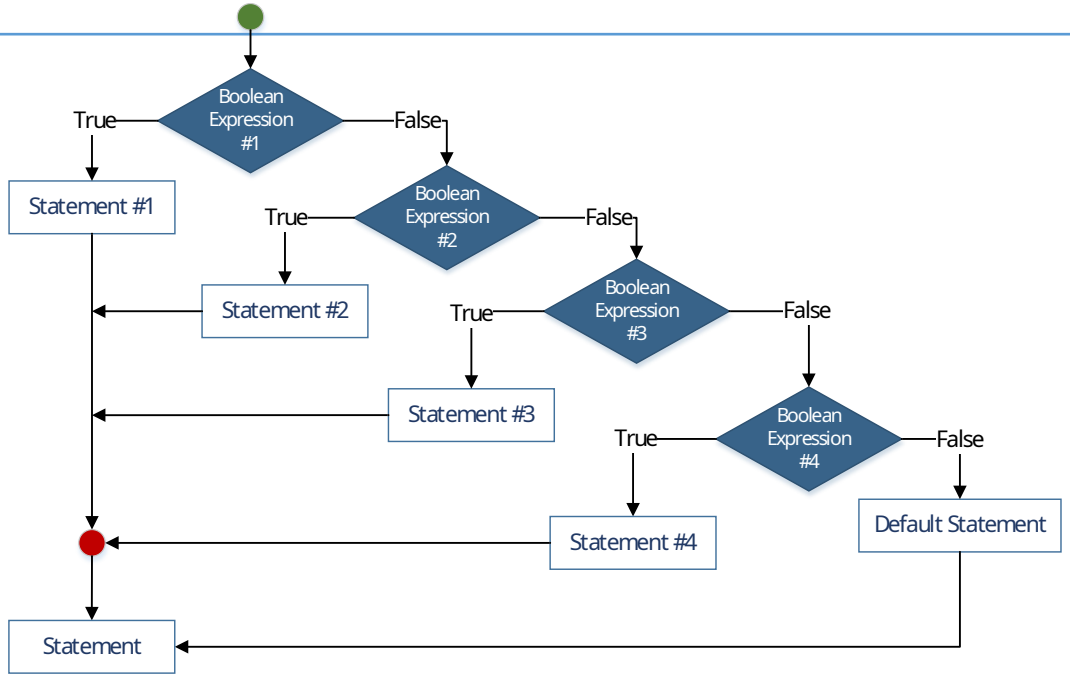
"The purpose of abstracting is not to be vague, but to create a new semantic level in which one can be absolutely precise" - **Edsger Dijkstra**



Abstractions => Polymorphism [Virtual Calls] => Runtime Cost

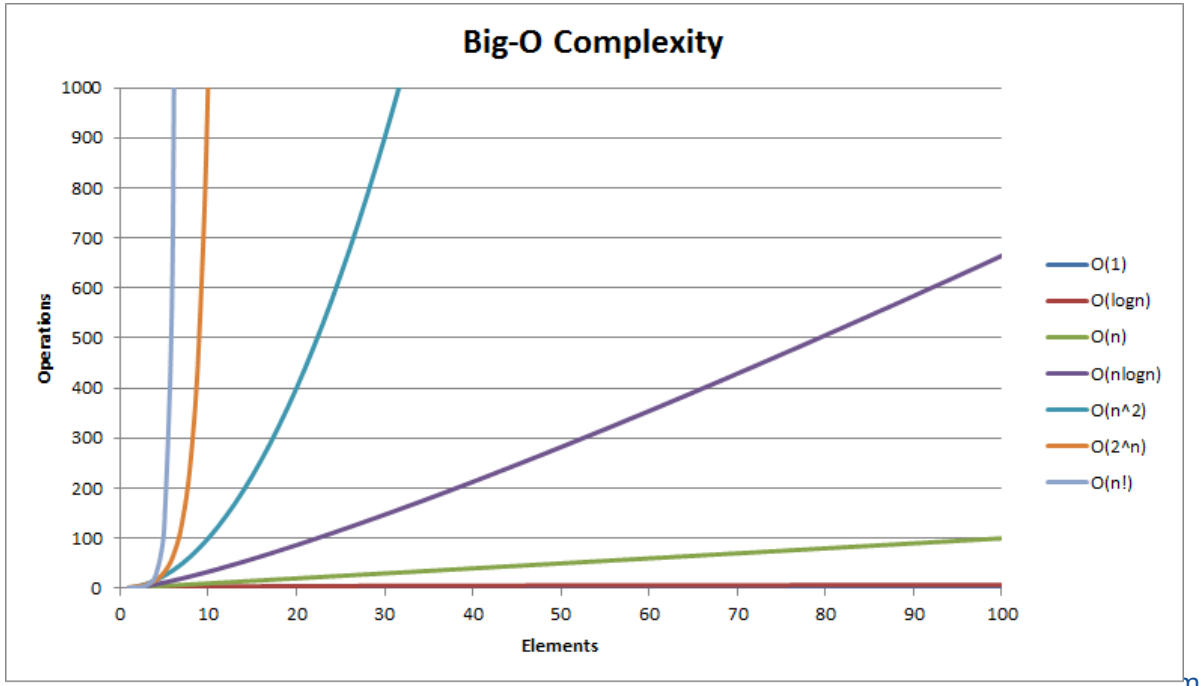
Cyclomatic Complexity

Cyclomatic complexity is the number of linearly independent paths through a program's source code.



Higher Cyclomatic Complexity => Branch Prediction Hell

Algorithms Complexity



[source: https://stackoverflow.com/questions/29927439/]

Algorithms are key to Service Time. Magnitude of N?

Algorithms Complexity

But .. is it all about Big-O Complexity?

Matrix Multiplication

Classical vs. Strassen Algorithm

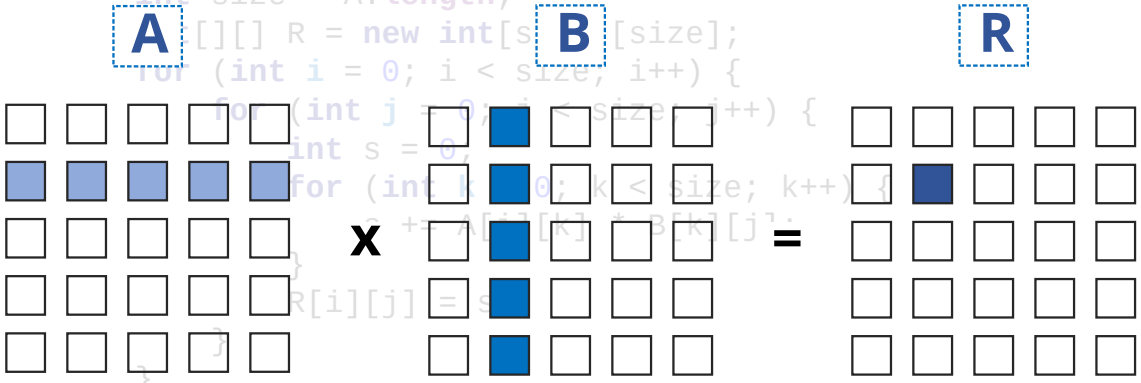
Algorithms Complexity

Matrix Multiplication – **Classical** (**ijk**) Algorithm

```
public int[][] multiply(int[][] A, int[][] B) {
    int size = A.length;
    int[][] R = new int[size][size];
    for (int i = 0; i < size; i++) {
        for (int j = 0; j < size; j++) {
            int s = 0;
            for (int k = 0; k < size; k++) {
                s += A[i][k] * B[k][j];
            }
            R[i][j] = s;
        }
    }
    return R;
}
```

Algorithms Complexity

Matrix Multiplication – **Classical** (**ijk**) Algorithm



$O(N^3)$

Algorithms Complexity

Matrix Multiplication – **Strassen** Algorithm

WIKIPEDIA
The Free Encyclopedia

Main page
Contents
Featured content
Current events
Random article
Donate to Wikipedia
Wikipedia store

Interaction

Help
About Wikipedia
Community portal
Recent changes
Contact page

Article Talk | Read Edit View history | Search Wikipedia

Strassen algorithm

From Wikipedia, the free encyclopedia

Not to be confused with the Schönhage–Strassen algorithm for multiplication of polynomials.

In linear algebra, the **Strassen algorithm**, named after Volker Strassen, is an algorithm for matrix multiplication. It is faster than the standard matrix multiplication algorithm and is useful in practice for large matrices, but would be slower than the fastest known algorithms for extremely large matrices.

Strassen's algorithm works for any ring, such as plus/multiply, but not all semirings, such as min/plus or boolean algebra, where the naive algorithm still works, and so called combinatorial matrix multiplication.

Contents [hide]

1 History
2 Algorithm
3 Asymptotic complexity
  3.1 Rank or bilinear complexity
  3.2 Cache behavior

$$O(N^{2.8074})$$

Algorithms Complexity

Classical (**ijk**) vs. **Strassen** Algorithm

Matrix size	**Classical (ijk)** (sec / op)	**Strassen** (sec / op)	Classical (ikj) (sec / op)
512 x 512	0.44	0.15	0.04
2,048 x 2,048	23	6.1	3
8,192 x 8,192	3,458	301	285
	$O(N^3)$	$O(N^{2.8074})$	$O(N^3)$

lower is better

Can we improve Classical (ijk) Algorithm to be faster?

Keeping the same Big-O Complexity

Algorithms Complexity

Matrix Multiplication – **Classical** (**ijk**) Algorithm

```
public int[][] multiply(int[][] A, int[][] B) {
    int size = A.length;
    int[][] R = new int[size][size];
    for (int i = 0; i < size; i++) {
        for (int j = 0; j < size; j++) {
            int s = 0;
            for (int k = 0; k < size; k++) {
                s += A[i][k] * B[k][j];
            }
            R[i][j] = s;
        }
    }
    return R;
}
```

Load misses
-CPU stalls-

$O(N^3)$

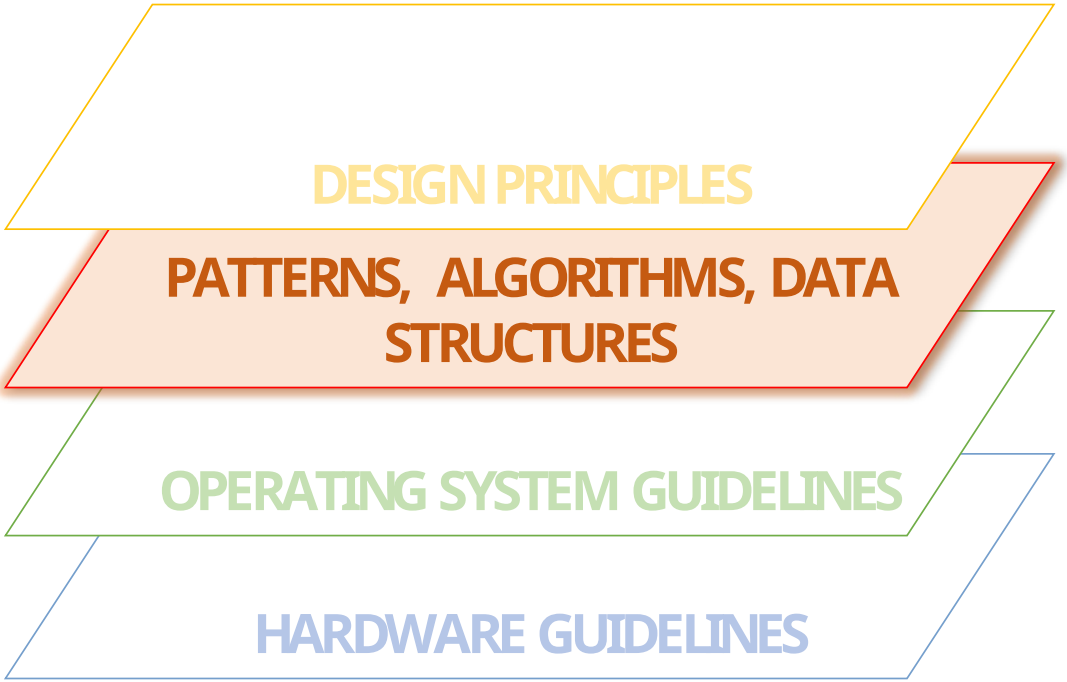
Algorithms Complexity

Classical (**ikj**) vs. **Strassen** Algorithm

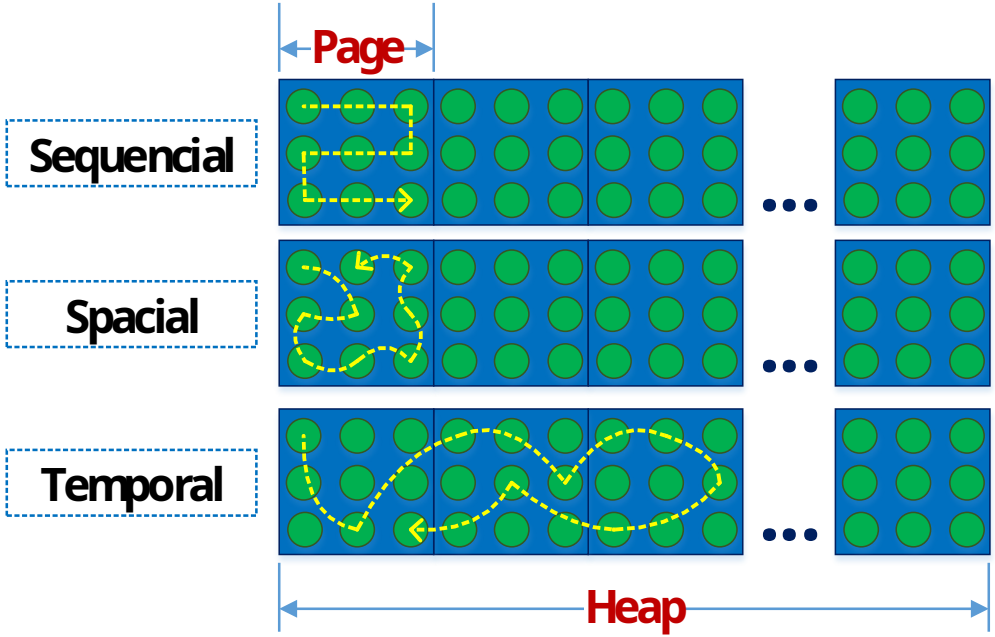
Matrix size	**Classical (ijk)** (sec / op)	**Strassen** (sec / op)	**Classical (ikj)** (sec / op)
512 x 512	0.44	0.15	0.04
2,048 x 2,048	23	6.1	3
8,192 x 8,192	3,458	301	285
	$O(N^3)$	$O(N^{2.8074})$	$O(N^3)$

lower is better

It is not all about Big-O Complexity, but also Memory Access Patterns !


DESIGN PRINCIPLES
PATTERNS, ALGORITHMS, DATA STRUCTURES
OPERATING SYSTEM GUIDELINES
HARDWARE GUIDELINES

Memory Access Patterns



Sequential - memory access is likely to follow a predictable pattern

Spatial - adjacent memory is likely to be required soon

Temporal - memory accessed recently will likely be required again soon

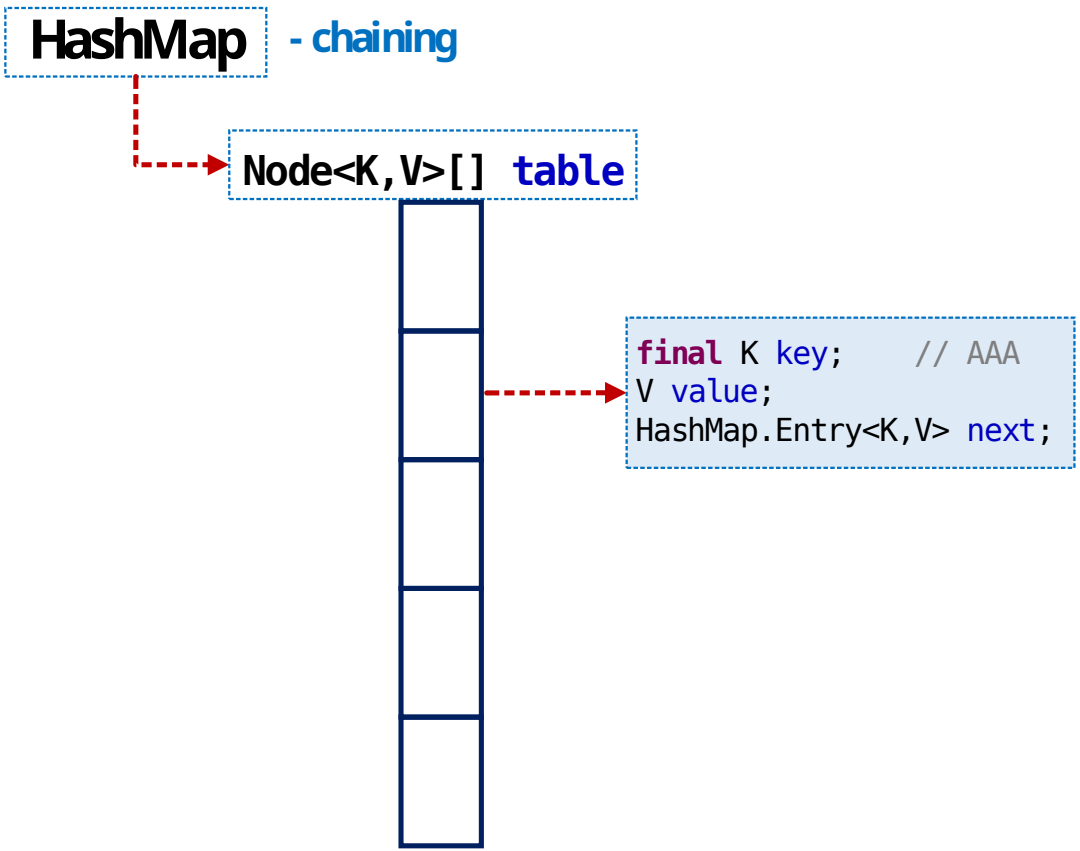
Pattern	Response Time (ns / op)
Sequential	0.97
Spatial	4.40
Temporal	37.34

HashMap

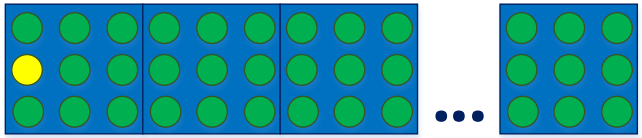
Chaining vs. Open Addressing

Data Structures

HashMap - chaining

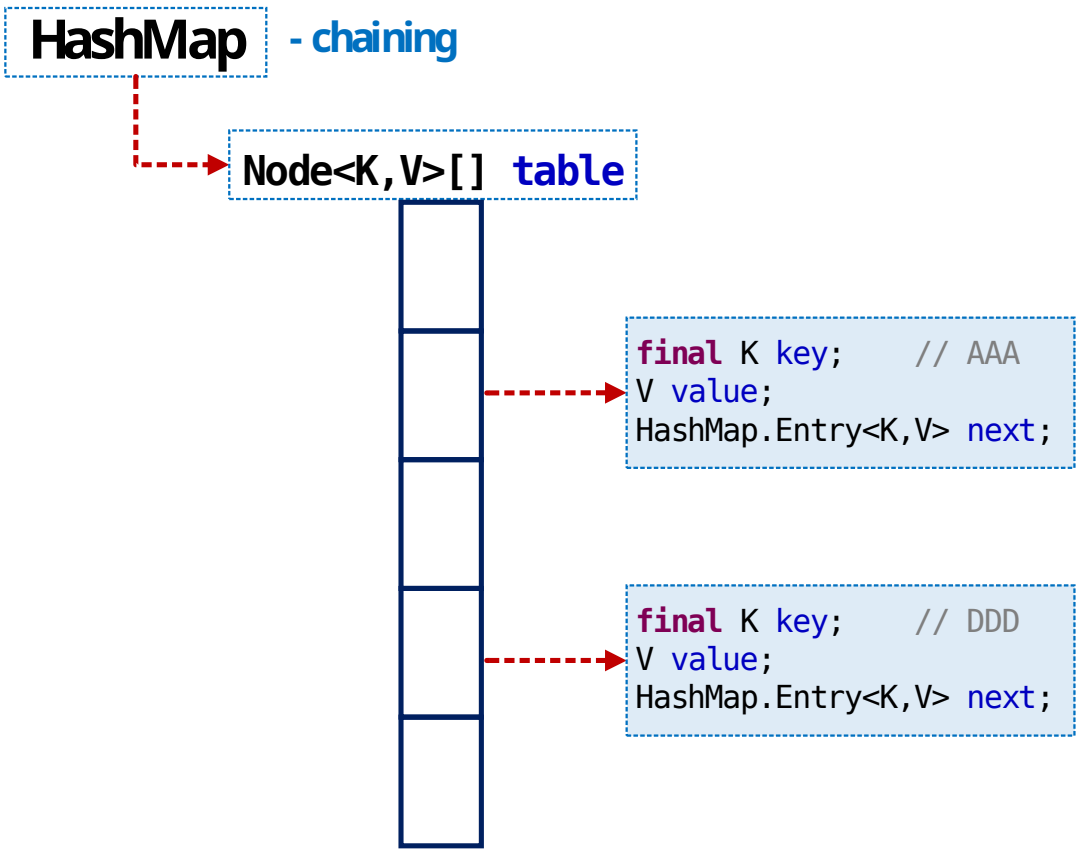


Memory Layout

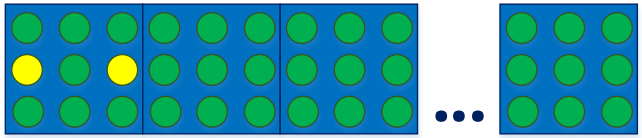


Data Structures

HashMap - chaining

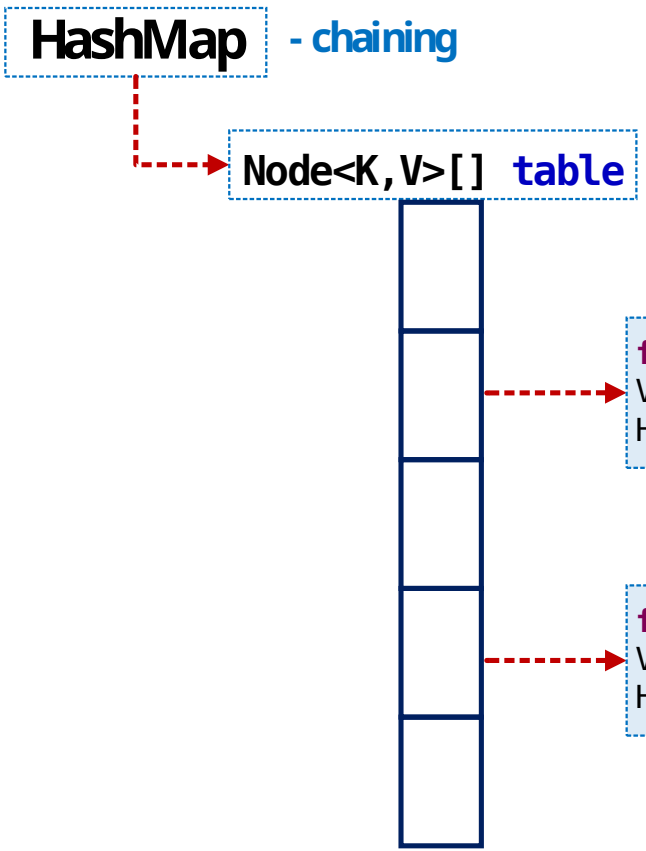


Memory Layout

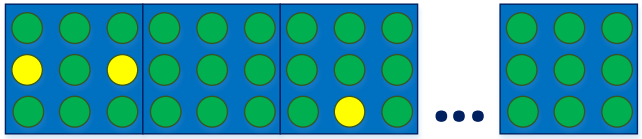


Data Structures

HashMap - chaining



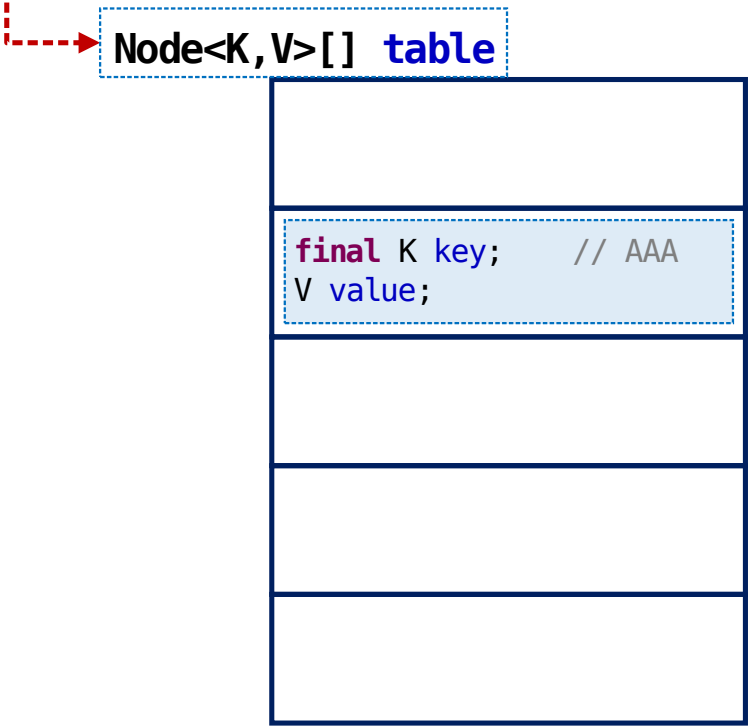
Memory Layout



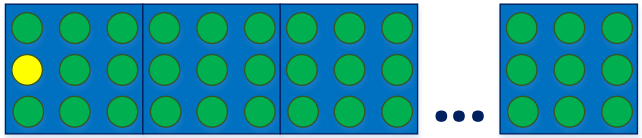
Collisions – **Temporal** memory access

Data Structures

HashMap - open addressing

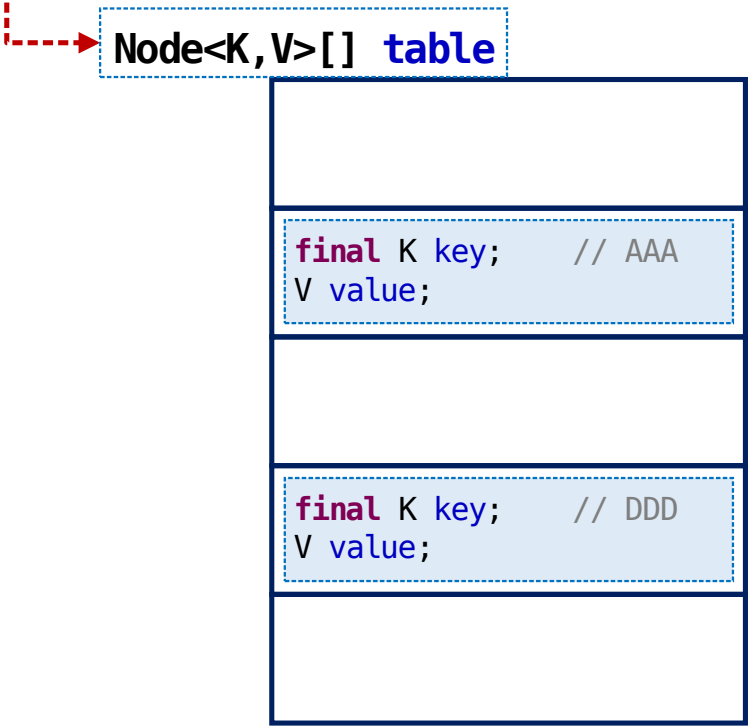


Memory Layout

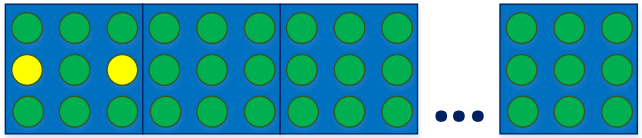


Data Structures

HashMap - open addressing



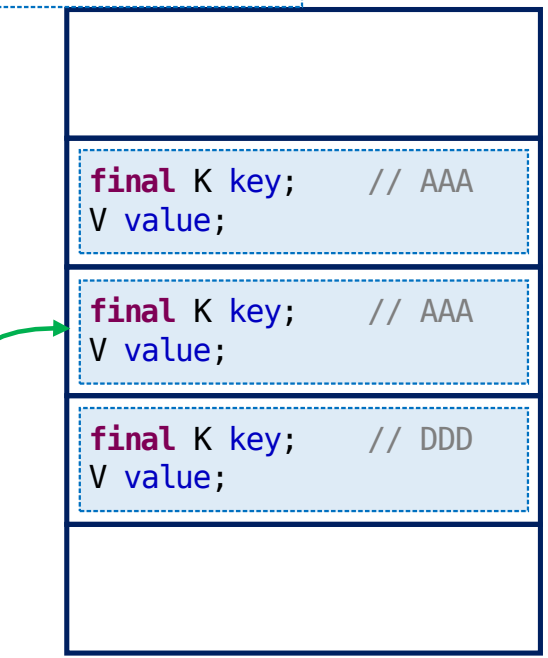
Memory Layout



Data Structures

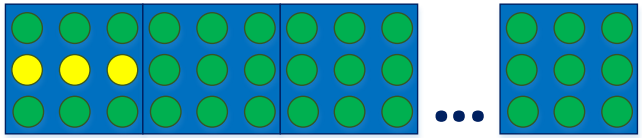
HashMap - open addressing

`Node<K,V>[] table`



collision resolved by **open addressing** with **linear probing** (internal = 1)

Memory Layout



Collisions – **Sequential** memory access

Chaining vs. Open Addressing

	Chaining	Open Addressing
Collision resolution	Use external data structure	Using hash table itself
Memory waste	Pointer size overhead per entry	No overhead, better locality
Performance dependence on Load Factor	Directly proportional	Proportional to (LF) / (1 - LF)
Implementation	Simple	Quite tricky

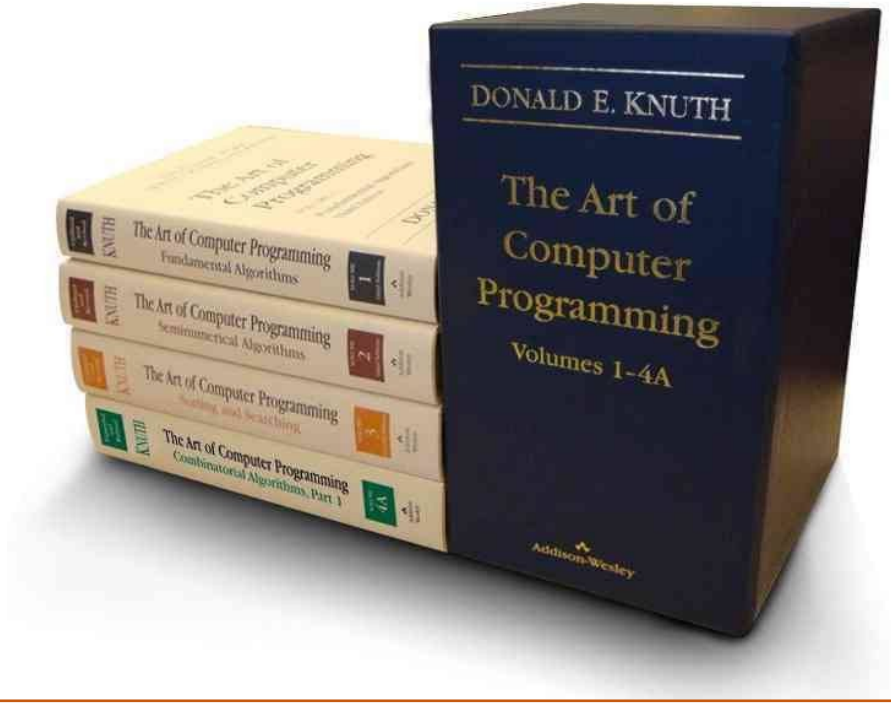
Open Addressing might be a performance alternative

Data Structures

*"... the **choice of data structures** often **makes a big difference**. Most programmers just grab whatever List they used last time rather than thinking which implementation is the right one ..."*

Martin Fowler

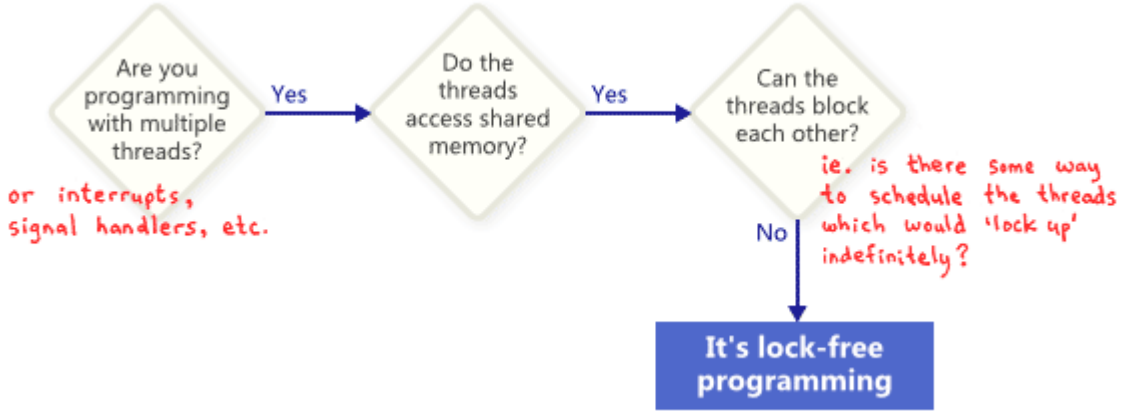
Data Structures



Oldies but Goldies!

Lock Free Algorithms

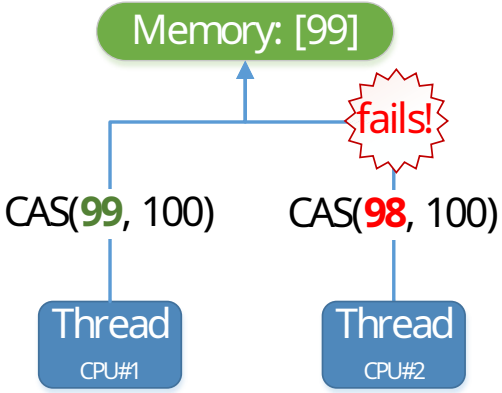
Lock Free - failure or suspension of any thread cannot cause failure or suspension of another thread and there is a guaranteed system-wide progress



[**Source**: http://preshing.com/20120612/an-introduction-to-lock-free-programming]

Lock Free Algorithms

Compare-And-Swap (CAS) - atomically update a memory location by another value if the previous value is the expected one




```
[lock] CMPXCHG reg, reg/mem
```

x86

x64

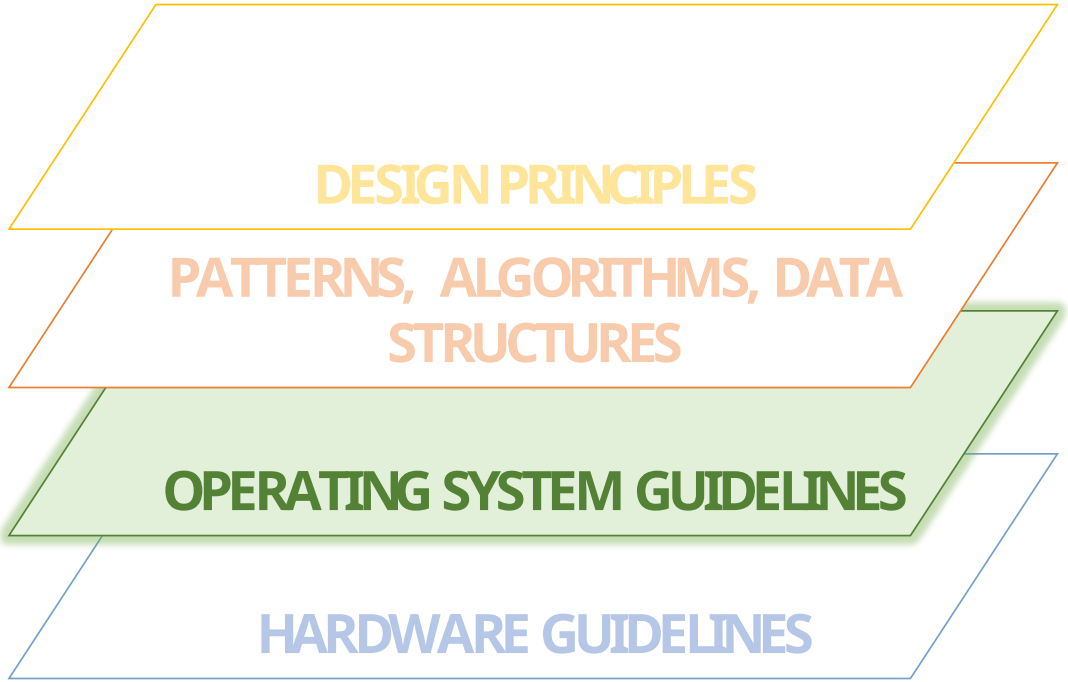
Lock Free Algorithms

Compare-And-Swap APIs

```
j.u.c.atomic.AtomicT
    public final boolean compareAndSet(T expect, T update)
```

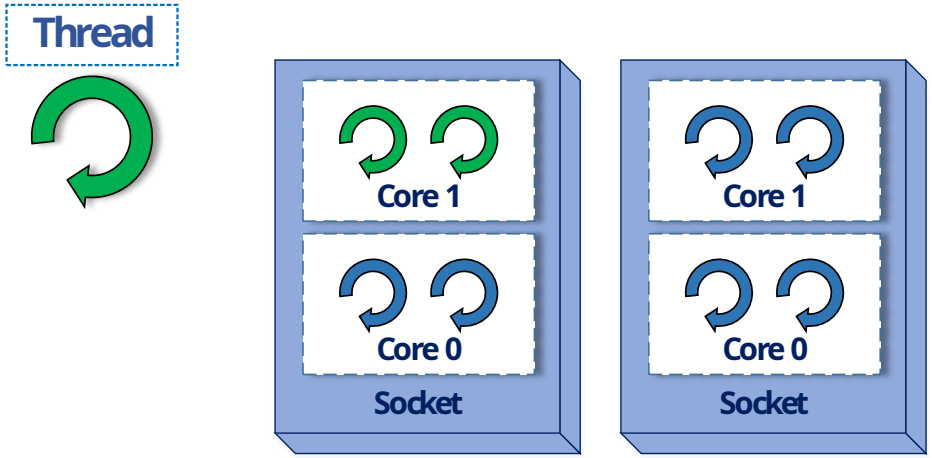
Data Structures using Compare-And-Swap

```
j.u.c.atomic.AtomicT
    public final T getAndIncrement()
    public final T getAndDecrement()
    public final T getAndAdd(T delta)
    public final T getAndSet(T newValue)

j.u.c.locks.ReentrantLock
    final void lock()
    protected final boolean tryAcquire(int acquires)

j.u.c.ConcurrentLinkedQueue
    boolean casItem(E cmp, E val)
    boolean casNext(Node<E> cmp, Node<E> val)
    final void updateHead(Node<E> h, Node<E> p)
    public boolean offer(E e)
    public boolean addAll(Collection<? extends E> c)
```


DESIGN PRINCIPLES
PATTERNS, ALGORITHMS, DATA STRUCTURES
OPERATING SYSTEM GUIDELINES
HARDWARE GUIDELINES

Thread Affinity

Thread Affinity binds a thread to a CPU or a range of CPUs so that the thread will execute only on the designated CPU or CPUs rather than any CPU



Thread affinity takes advantages on CPU cache memory

Thread Affinity

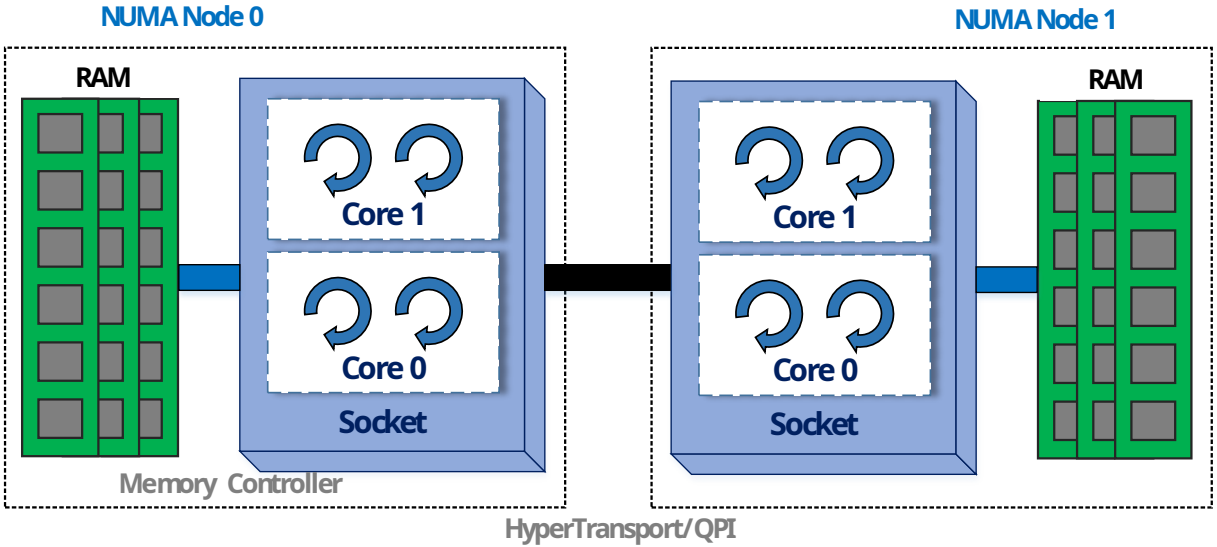
In Java

In Linux

taskset - retrieve or set processes's CPU affinity

NUMA

Non-Uniform-Memory-Access (NUMA) is a memory design where the memory access time depends on the memory location relative to the processor



NUMA-aware allocator has been implemented to take advantage of local memory

NUMA

In Java

-XX:+UseNUMA activates NUMA-aware collector

-XX:+UseParallelGC needs to be enabled as well

NUMA Performance Metrics

When evaluated against the SPEC JBB 2005 benchmark on an 8-chip Opteron machine, NUMA-aware systems showed the following performance increases:

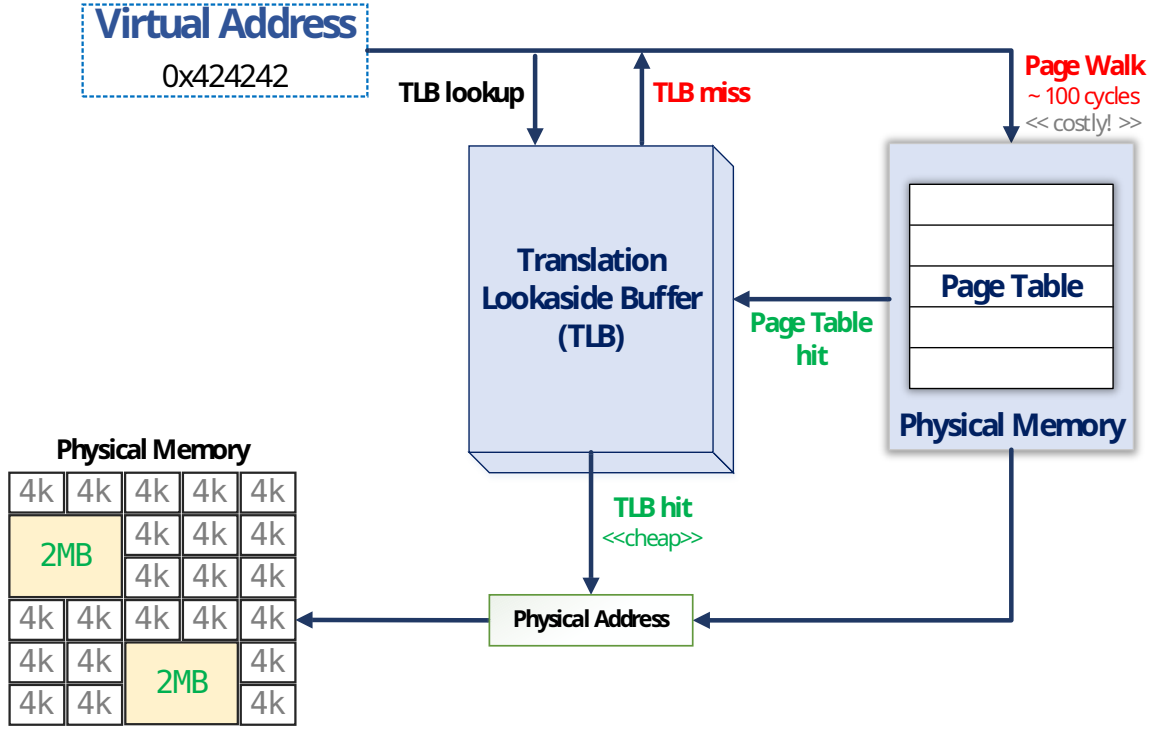
- 32 bit – About 30 percent increase in performance with NUMA-aware allocator
- 64 bit – About 40 percent increase in performance with NUMA-aware allocator

In Linux

numactl - control NUMA policy for processes or shared memory

Large Pages

Using Huge Pages the TLB can represent larger memory rage hence reduces TLB misses and the number of Page Walks



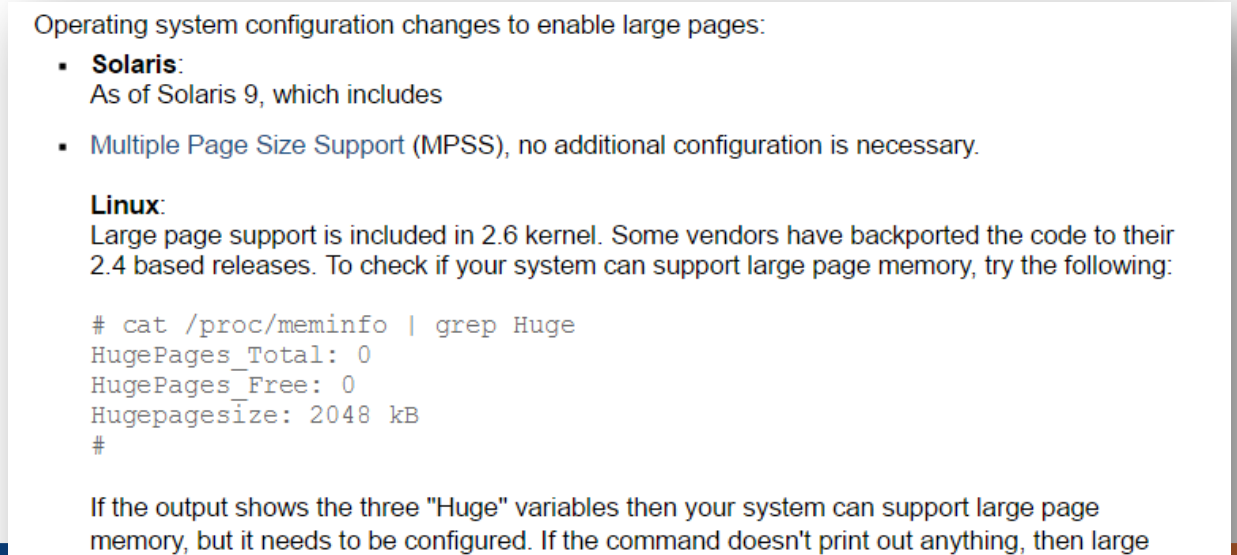
Large Pages

In Java

-XX:+UseLargePages but it needs Large Pages enabled in the OS

-XX:LargePageSizeInBytes specifies explicit large page size

-XX:+UseTransparentHugePages advices Java heap to use Transparent Huge Pages



Operating system configuration changes to enable large pages:

- **Solaris**:
  As of Solaris 9, which includes
- Multiple Page Size Support (MPSS), no additional configuration is necessary.

Linux:
Large page support is included in 2.6 kernel. Some vendors have backported the code to their 2.4 based releases. To check if your system can support large page memory, try the following:

```
# cat /proc/meminfo | grep Huge
HugePages_Total: 0
HugePages_Free: 0
Hugepagesize: 2048 kB
#
```

If the output shows the three "Huge" variables then your system can support large page memory, but it needs to be configured. If the command doesn't print out anything, then large

RamFS & TmpFS

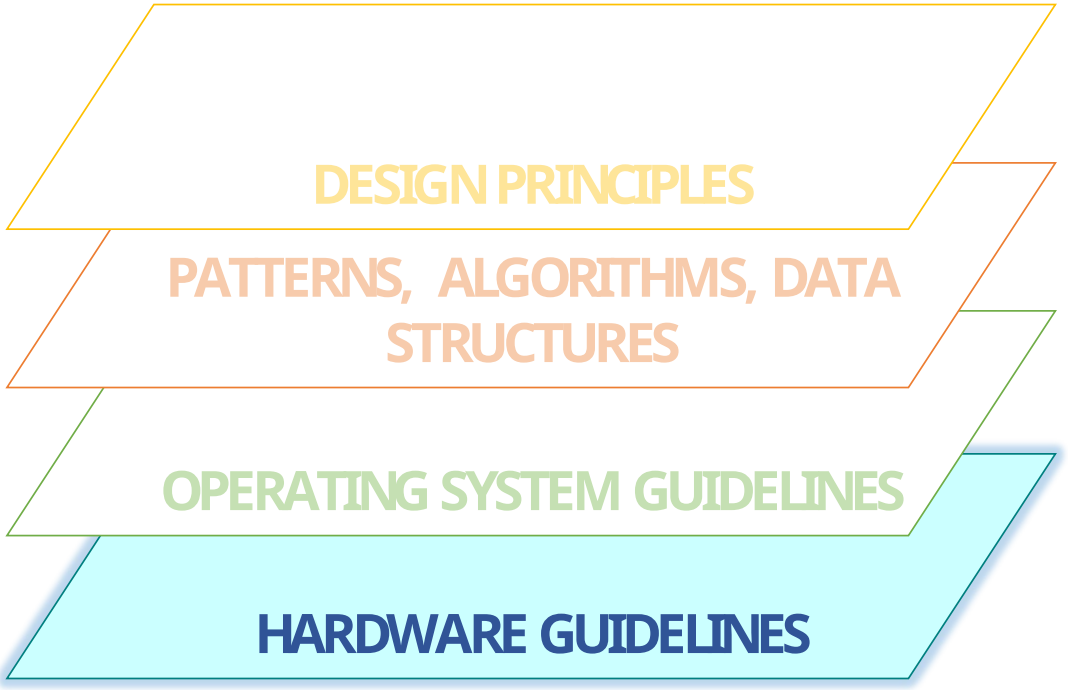
RamFS & TmpFS allocate a part of the physical memory to be used as a partition (e.g. write/read files).

Suitable for applications which performs a lot disk reads/writes (e.g. logging, auditing)

RamFS & TmpFS

	HDD		SSD		RAMFS	
	Read MB/s	Write MB/s	Read MB/s	Write MB/s	Read MB/s	Write MB/s
Seq	112.3	109.3	447.9	335.7	6,566	7,760
512K	41.69	48.05	402.7	348.9	6,749	7,172

higher is better


DESIGN PRINCIPLES
PATTERNS, ALGORITHMS, DATA STRUCTURES
OPERATING SYSTEM GUIDELINES
HARDWARE GUIDELINES

False Sharing

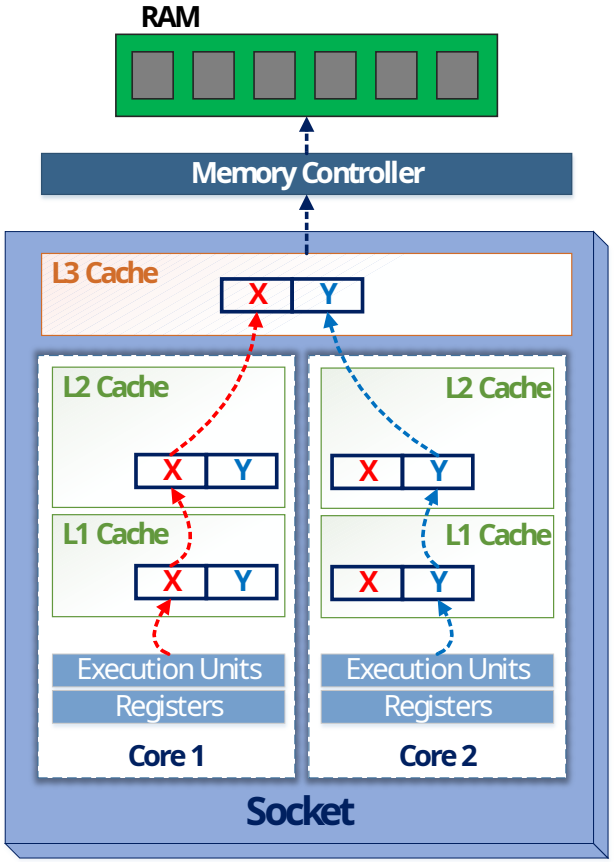
```
public class FalseSharing {
    public int X;
    public int Y;
}

FalseSharing sharedInstance = new FalseSharing();
```

Thread 1

```
void incrementX () {
    sharedInstance.X ++
}
```

Thread 2

```
void incrementY () {
    sharedInstance.Y ++
}
```



False Sharing

In Java

@Contended annotation pads fields so they will sit on different cache lines

```
public class FalseSharing {
    public int X;

    @sun.misc.Contended
    public int Y;
}
```

Threads	Baseline (# of ops / ms)	@Contended (# of ops / ms)
2	478.051	1,378.564

higher is better

C-States

C-States cut the clock signal and CPU power to save energy

but ... **more time is required for CPU to "wake up"** and again be 100% operational

Consider disabling C-States to maintain a steady high CPU performance

“Profiling is simple, all you have to know is everything”

Sergey Kuksenko – Java Performance Engineer @ Oracle

“Performance is simple, you just have to know everything”

Ionuţ Baloşin

THANK YOU

Ionuţ Baloşin

Software Architect
ibalosin@luxoft.com
@ionutbalosin

LUXOFT

LXFT LISTED NYSE

Bibliography

- http://mechanical-sympathy.blogspot/2012/08/memory-access-patterns-are-important.html
- http://mechanical-sympathy.blogspot/2011/07/false-sharing.html
- http://vanillajava.blogspot/2012/01/java-thread-affinity-support-for-hyper.html
- http://www.geeksforgeeks.org/hashing-set-3-open-addressing
- http://www.algolist.net/Data_structures/Hash_table/Open_addressing
- http://preshing.com/20120612/an-introduction-to-lock-free-programming
- http://heather.cs.ucdavis.edu/~matloff/50/PLN/lock.pdf
- http://www.glennklockwood.com/hpc-howtos/process-affinity.html
- http://docs.oracle.com/javase/7/docs/technotes/guides/vm/performance-enhancements-7.html
- http://cenalulu.github.io/linux/huge-page-on-numa
- http://www.thegeekstuff.com/2008/11/overview-of-ramfs-and-tmpfs-on-linux
- https://www.laptopmag.com/articles/faster-than-an-ssd-how-to-turn-extra-memory-into-a-ram-disk
- http://daniel.mitterdorfer.name/articles/2014/false-sharing

Appendix

LUXOFT

LXFT
LISTED
NYSE

Algorithms Complexity

Matrix Multiplication – **Classical** (**ikj**) Algorithm

```
public int[][] multiplyIKJ(int[][] A, int[][] B) {
    int size = A.length;
    int[][] R = new int[size][size];
    for (int i = 0; i < size; i++) {
        for (int k = 0; k < size; k++) {
            int aik = A[i][k];
            for (int j = 0; j < size; j++) {
                R[i][j] += aik * B[k][j];
            }
        }
    }
    return R;
}
```

$O(N^3)$